

Generative AI Project:

CoverComposer: AI-Powered Music Track Generator

Project Description:

It is an AI-powered web application that combines mood-based music recommendation, facial emotion detection, AI-generated album artwork, and user personalization into a single interactive platform.

The application allows users to:

- Login and manage accounts securely.
- Detect emotions through facial expressions using Face API.
- Select moods manually.
- Listen to mood-specific music tracks.
- Generate AI-powered album covers using Google Gemini.
- Store listening history and generated artwork.
- Explore previously generated covers through an art gallery.

Scenario:

In today's digital world, users often spend a significant amount of time searching for music that matches their current mood. Traditional music applications require users to manually browse playlists, which may not always reflect their emotional state. Additionally, creating personalized album artwork requires design skills and specialized software.

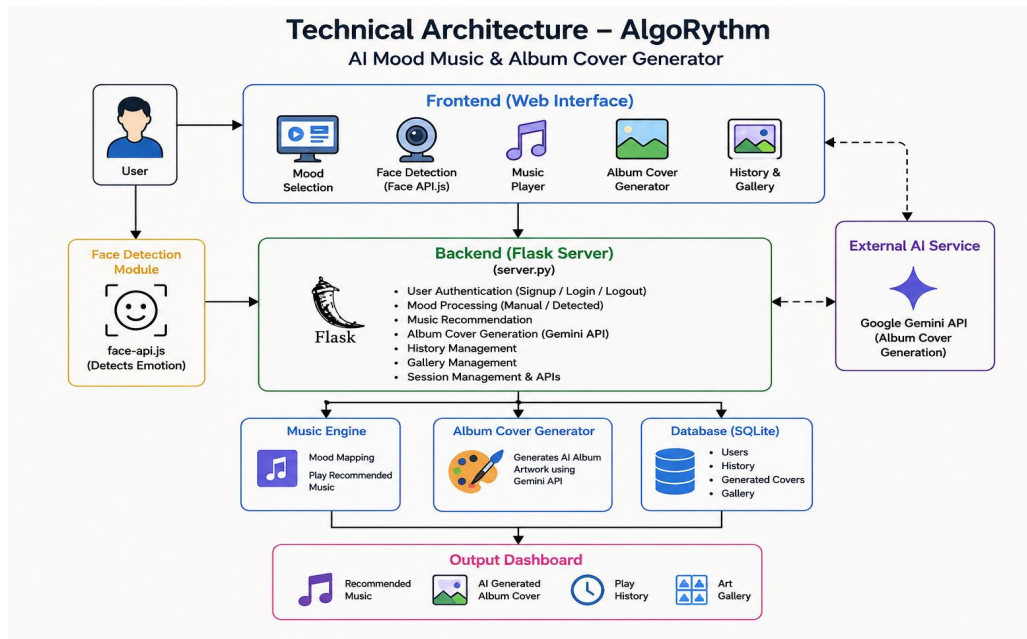
The AlgoRythm system addresses these challenges by integrating Artificial Intelligence, facial emotion detection, mood-based music recommendation, and AI-generated album cover creation into a single platform.

When a user accesses the application, they can either manually select a mood or allow the system to detect their emotion through facial expression analysis using a webcam. The detected mood is sent to the Flask backend, where the system recommends suitable music tracks. Simultaneously, the application uses Google Gemini AI to generate a unique album cover based on the selected mood and music theme.

The system stores user information, generated artwork, and listening history in a SQLite database, allowing users to revisit their previous activities through the History and Gallery modules.

This intelligent workflow provides a personalized entertainment experience by combining emotion recognition, music recommendation, and AI-generated visual content, making the application more engaging and user-friendly than traditional music platforms.

Technical Architecture:



Pre-requisites:

The system follows a three-layer architecture consisting of the Frontend Layer, Backend Layer, and AI Services Layer.

1. User Layer

The user interacts with the application through a web browser. Users can manually select moods, detect emotions using a webcam, listen to music, and generate AI-powered album covers.

2. Frontend Layer

The frontend is developed using HTML, CSS, and JavaScript. It provides an interactive dashboard where users can:

- Select moods

- Detect emotions through facial expressions
- Play music
- Generate album covers
- View history and gallery sections

3. Emotion Detection Module

Face API.js analyzes facial expressions captured through the webcam and predicts emotions such as Happy, Sad, Calm, or Energetic. The detected emotion is forwarded to the backend for processing.

4. Flask Backend Server

The Flask server acts as the central controller of the application. It processes user requests, manages sessions, communicates with AI services, and retrieves data from the database.

5. Music Recommendation Engine

Based on the detected or selected mood, the system maps the emotion to predefined playlists and recommends suitable music tracks.

6. AI Album Cover Generator

The backend communicates with the Google Gemini API to generate creative album cover concepts corresponding to the selected mood and music theme.

7. Database Layer

SQLite is used to store:

- User account details
- Login credentials
- Music listening history
- Generated album cover information

8. Output Dashboard

The final results are displayed on the dashboard, including:

- Recommended music tracks
- AI-generated album artwork
- User history
- Art gallery records

Project Workflow:

Activity 1: User Authentication

The first stage of the system is user authentication, which ensures secure access to the application. This module manages user accounts and maintains personalized sessions.

Key Functions:

1. User Registration: New users create an account by providing their details.
2. Login Validation: The system verifies user credentials before granting access.
3. Session Management: Active user sessions are maintained for a seamless experience.
4. Data Security: User information is securely stored and protected.

The authentication process helps prevent unauthorized access and enables personalized recommendations based on individual user preferences.

Activity 2: Mood Detection

After logging in, the user can access the mood detection feature. The system uses the device camera and AI-based emotion recognition techniques to identify the user's current emotional state.

Key Functions:

1. Camera Access: Captures the user's facial image in real time.
2. Face Detection: Identifies and extracts the face from the captured frame.
3. Emotion Recognition: Analyzes facial expressions using AI models.
4. Mood Classification: Categorizes emotions into moods such as Happy, Sad, Angry, Surprise, Neutral, and Fear.

The detected mood serves as the primary input for generating personalized music recommendations.

Activity 3: Music Recommendation

Based on the identified mood, the system recommends suitable songs and playlists. This feature aims to enhance the user's listening experience through personalization.

Key Functions:

1. Mood Mapping: Associates detected emotions with music categories.
2. Song Recommendation: Selects tracks that match the user's emotional state.
3. Music Playback: Plays the recommended songs through the built-in player.
4. Visualizer Effects: Displays animated visual effects during playback.

This module ensures that users receive music that aligns with their current feelings and preferences.

Activity 4: AI Cover Generation

The application includes an AI-powered cover generation feature that creates artwork based on user moods and prompts.

Key Functions:

1. Mood Input: Accepts mood-related input from the user.
2. Artwork Generation: Uses Gemini AI to generate creative cover concepts.
3. Image Creation: Produces unique visual artwork reflecting the selected mood.
4. Gallery Storage: Saves generated covers for future access.

This feature adds a creative dimension to the application by transforming emotions into visual representations.

Activity 5: History Management

The history management module stores and organizes user activities for future reference.

Key Functions:

1. Save Listening Activity: Records previously played songs and playlists.
2. Store Generated Covers: Maintains a collection of AI-generated artwork.
3. Mood Tracking: Saves detected emotions for analysis.
4. Retrieve Records: Allows users to access their past activities and content.

This functionality improves personalization and enables users to review their previous interactions with the system.

Workflow Outcome

The workflow integrates authentication, mood detection, music recommendation, AI cover generation, and history management into a single intelligent platform. By combining artificial intelligence, facial emotion recognition, and personalized content generation, the system delivers an engaging and user-centric music experience.

Milestone 1: Model Architecture

Name	Type	Compressed size	Password p...	Size	Ratio	Date modified
covers	File folder					12-06-2026 11:21
models	File folder					12-06-2026 10:37
songs	File folder					12-06-2026 10:37
auth.js	JavaScript Source File	2 KB	No	4 KB	68%	12-06-2026 10:45
firebase.js	JavaScript Source File	1 KB	No	1 KB	38%	10-06-2026 14:59
index.html	Microsoft Edge HTML Do...	3 KB	No	10 KB	70%	12-06-2026 10:57
login.html	Microsoft Edge HTML Do...	1 KB	No	2 KB	64%	12-06-2026 10:38
script.js	JavaScript Source File	7 KB	No	23 KB	73%	12-06-2026 10:57
style.css	CSS Source File	4 KB	No	20 KB	80%	12-06-2026 10:57
welcome.html	Microsoft Edge HTML Do...	2 KB	No	4 KB	71%	12-06-2026 10:38

Name	Type	Compressed size	Password p...	Size	Ratio	Date modified
__pycache__	File folder					12-06-2026 10:56
audio_generator.py	Python File	2 KB	No	4 KB	59%	12-06-2026 11:26
cover_generator.py	Python File	4 KB	No	9 KB	65%	12-06-2026 10:56
database.db	Data Base File	2 KB	No	24 KB	95%	12-06-2026 11:21
database.py	Python File	2 KB	No	4 KB	74%	12-06-2026 10:37
server.py	Python File	2 KB	No	5 KB	71%	12-06-2026 10:56

index.html

Provides:

- Login Interface
- Signup Interface
- Mood Selection Dashboard
- Music Recommendation Section
- Album Cover Generation Controls
- History and Gallery Navigation

script.js

Responsible for:

- API Communication
- User Interaction Handling
- Mood Processing
- Dynamic Content Updates
- Music Recommendation Display
- Album Cover Display

style.css

Provides:

- Responsive Layout Design
- Dashboard Styling
- Button Styling
- Navigation Components
- Gallery Layout
- Mobile-Friendly Interface

Milestone: Backend Development

Import Required Modules

```
from flask import Flask, render_template, request, jsonify, session
from flask_cors import CORS
import sqlite3
import os
import json
from datetime import datetime

from database import init_db
from cover_generator import generate_album_cover
```

Flask & Request Handling

Flask, request, jsonify, render_template, and session are used for creating application routes, processing user requests, managing sessions, and sending responses.

Database Management

SQLite3 is used for database operations including user registration, login verification, history storage, and gallery management.

AI Integration

Google Gemini AI libraries are used to generate personalized album cover artwork based on user-selected moods.

Utility Modules

os, json, datetime, and other utility modules are used for file handling, data processing, and timestamp generation.

Initialize Flask Application and Paths

```
app = Flask(__name__)
app.secret_key = "algorithmm_secret_key"

CORS(app)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

DATABASE = os.path.join(BASE_DIR, "algorithmm.db")

init_db()
```

BASE_DIR

Represents the root directory of the project.

DATABASE_PATH

Stores the SQLite database file used by the application.

TEMPLATE_FOLDER

Contains HTML templates used for rendering web pages.

STATIC_FOLDER

Contains CSS, JavaScript, images, and frontend assets.

Setup Database Handling

```
def get_db_connection():
    conn = sqlite3.connect("algorith.db")
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_db_connection()

    conn.execute("""
        CREATE TABLE IF NOT EXISTS users(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE,
            password TEXT
        )
    """)

    conn.commit()
    conn.close()
```

Database Initialization

Creates all required database tables during application startup.

User Table

Stores: User ID, Username, Password

History Table

Stores: Selected Mood, Recommended Music, Generated Album Covers, Timestamp

Gallery Table

Stores:

Generated Cover Images

User Records

Cover Metadata

Configure Gemini AI

```
import google.generativeai as genai

genai.configure(
    api_key=GEMINI_API_KEY
)

model = genai.GenerativeModel(
    "gemini-pro"
)
```

Gemini AI is integrated to generate unique album cover designs according to user mood and music preferences.

Features

API Configuration

Prompt Creation

Artwork Generation

AI Response Processing

User Authentication Module

```
@app.route('/signup', methods=['POST'])
def signup():

    username = request.form['username']
    password = request.form['password']

    conn = get_db_connection()

    conn.execute(
        "INSERT INTO users(username,password) VALUES (?,?)",
        (username,password)
    )

    conn.commit()
    conn.close()

    return jsonify({"message": "Signup Successful"})
```

Signup Functionality

Allows new users to register and create accounts.

Login Functionality

Validates user credentials stored in the database.

Session Management

Maintains active user sessions throughout application usage.

Mood Detection and Processing

Mood Categories

Happy, Sad, Calm, Energetic

The backend receives mood information either from manual selection or facial emotion detection and processes it for recommendation generation.

Music Recommendation Engine

```
function recommendMusic(mood){  
  
  const playlists = {  
    happy: "Happy Playlist",  
    sad: "Sad Playlist",  
    calm: "Calm Playlist",  
    energetic: "Workout Playlist"  
  };  
  
  return playlists[mood];  
}
```

Recommendation Process

Receives user mood

Maps mood to suitable playlists

Generates personalized music recommendations

Returns recommendations to the frontend

This enables users to quickly access music that matches their emotional state.

AI Album Cover Generation

```
def generate_album_cover(mood):  
  
    prompt = f"""  
    Create an artistic album cover  
    representing the mood:  
    {mood}  
    """  
  
    response = model.generate_content(  
        prompt  
    )  
  
    return response.text
```

Album Cover Generation Process

- User selects or detects a mood
- Mood data is sent to Flask backend
- Gemini AI generates album cover content
- Generated cover is processed and displayed
- Cover information is saved in the gallery

This creates personalized visual artwork that matches the user's mood and music preferences.

History Management

Tracks all user activities including:

- Selected Moods
- Recommended Music
- Generated Covers
- Date and Time Information

Users can view previous activities through the History section.

Gallery Management

Stores and manages generated album covers.

Features

Artwork Storage

Cover Retrieval

Gallery Display

User Collection Management

The gallery allows users to revisit previously generated album cover designs.

```
@app.route('/history')
def history():

    conn = get_db_connection()

    history = conn.execute(
        "SELECT * FROM history"
    ).fetchall()

    conn.close()

    return jsonify(history)
```

Homepage Route

Serves the main dashboard page containing:

Login Interface

Signup Interface

Mood Selection Controls

Music Recommendation Section

Album Cover Generator

Gallery Access

History Access

Album Cover Generation Route

Accepts User Input

Mood Selection

User Preferences

Executes

Mood Processing

Gemini AI Request

Cover Generation

Database Storage

Returns

Generated Album Cover

Recommendation Results

Updated User Dashboard

History Retrieval Endpoint

Retrieves stored user activities from the database and displays them within the History section.

Gallery Retrieval Endpoint

Retrieves previously generated album covers and displays them in the Gallery section.

Front End Dashboard Structure

```
<div class="dashboard">

  <h1>AlgoRythm</h1>

  <select id="moodSelect">
    <option>Happy</option>
    <option>Sad</option>
    <option>Calm</option>
    <option>Energetic</option>
  </select>

  <button id="generateBtn">
    Generate Cover
  </button>

</div>
```

Face Detection Logic

- **async function detectEmotion():** An asynchronous function wrapper used to handle the non-blocking nature of machine learning inference.
- **const detections:** A constant variable that stores the result of the face detection process.
- **await:** A keyword that pauses the function execution until the promise from the faceapi library is resolved.
- **faceapi:** The global object representing the face-api.js library.
- **.detectSingleFace(video):** A method that identifies the primary face in the provided video HTML element.
- **.withFaceExpressions():** A chainable method that instructs the model to perform emotion classification on the detected face.
- **return detections.expressions:** The statement that sends the final object containing probability scores for various emotions back to the caller.

```
async function detectEmotion(){  
  
    const detections =  
        await faceapi  
            .detectSingleFace(video)  
            .withFaceExpressions();  
  
    return detections.expressions;  
}
```

Milestone: Deployment

In Milestone 5, the focus is on deploying the CoverComposer application on a local system. This involves configuring the server environment, ensuring all dependencies are installed and launching the app locally to simulate a real-world setup. This helps verify that the app can run smoothly in a stable environment, allowing you to fine-tune any functionality before deploying it to a cloud platform in the future.

cd (folder_location)

pip install -r requirements.txt

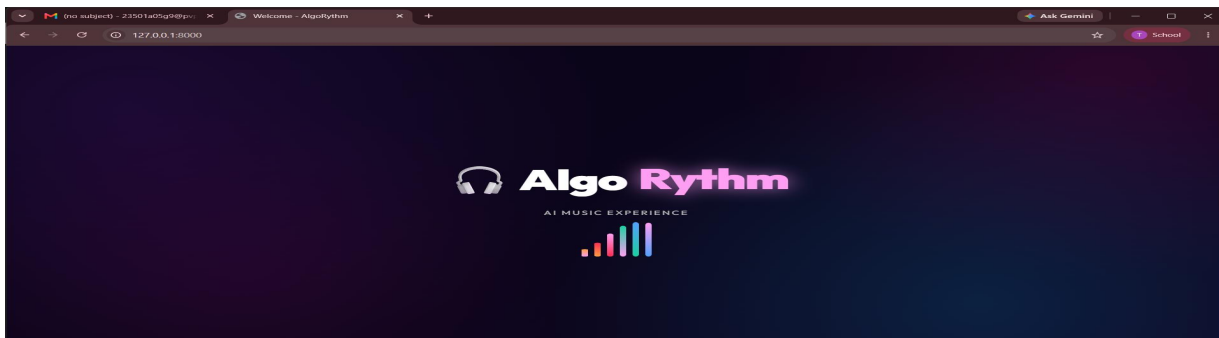
uvicorn main:app --reload

This installed the prerequisite packages and launches the application on

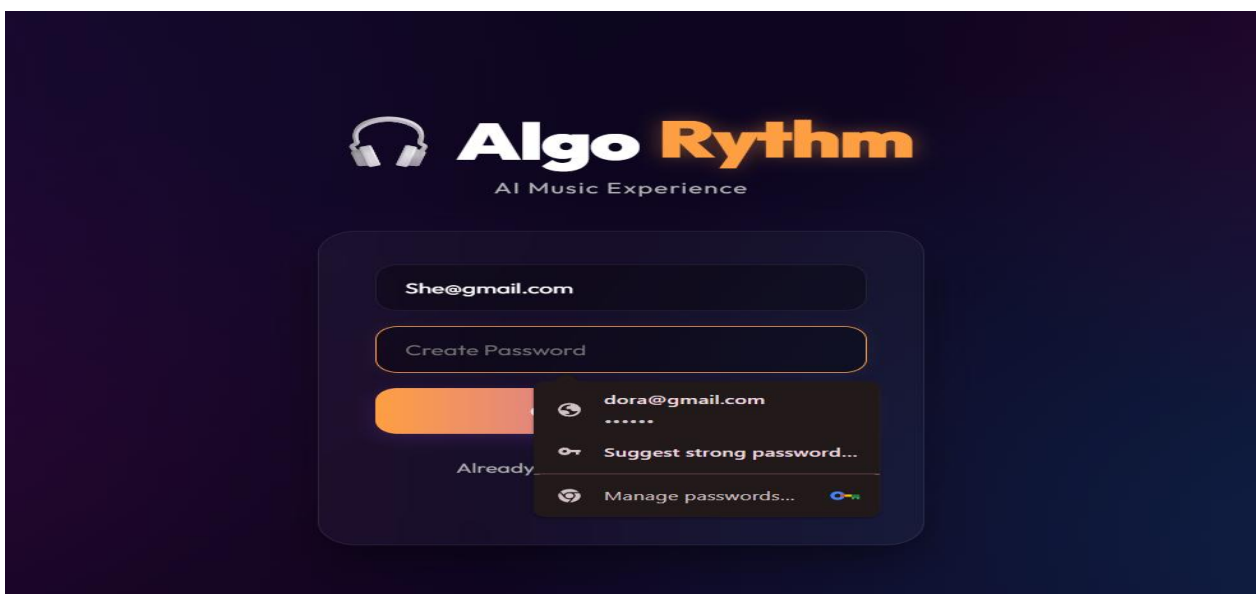
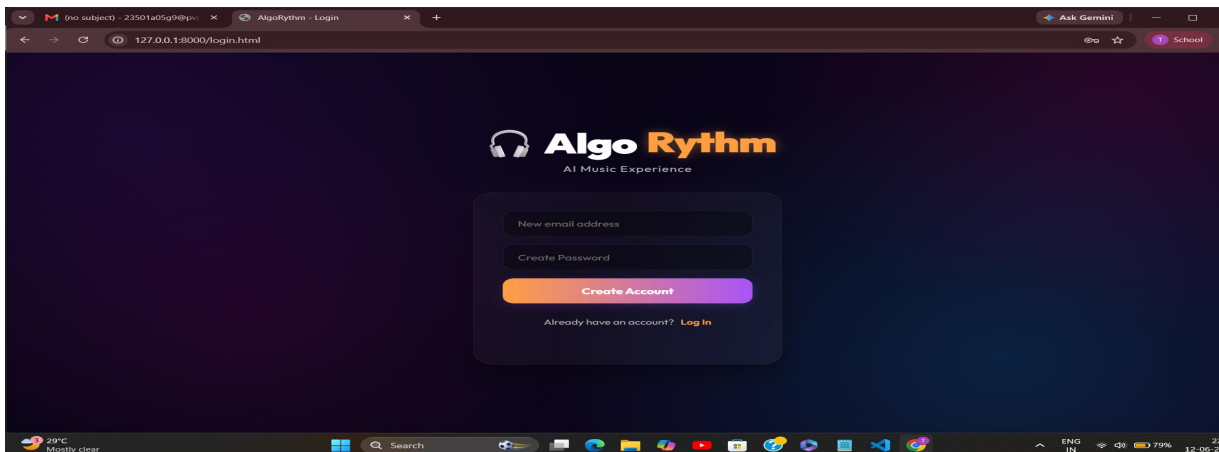
<http://127.0.0.1:8000>

Exploring website's Web Pages:

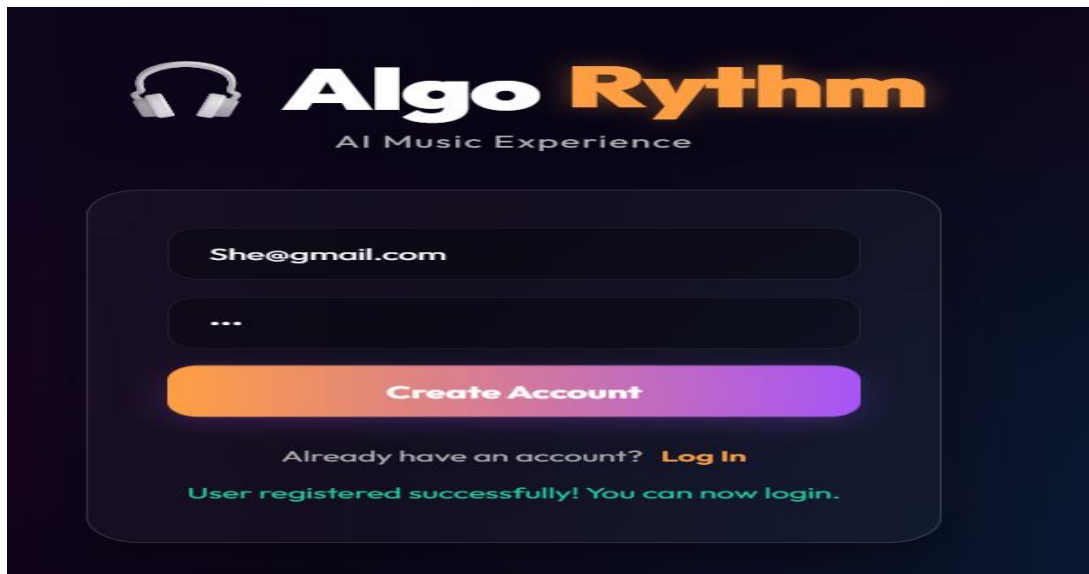
Welcome Page:



Login Page:



Create Account



Algo Rythm
AI Music Experience

She@gmail.com

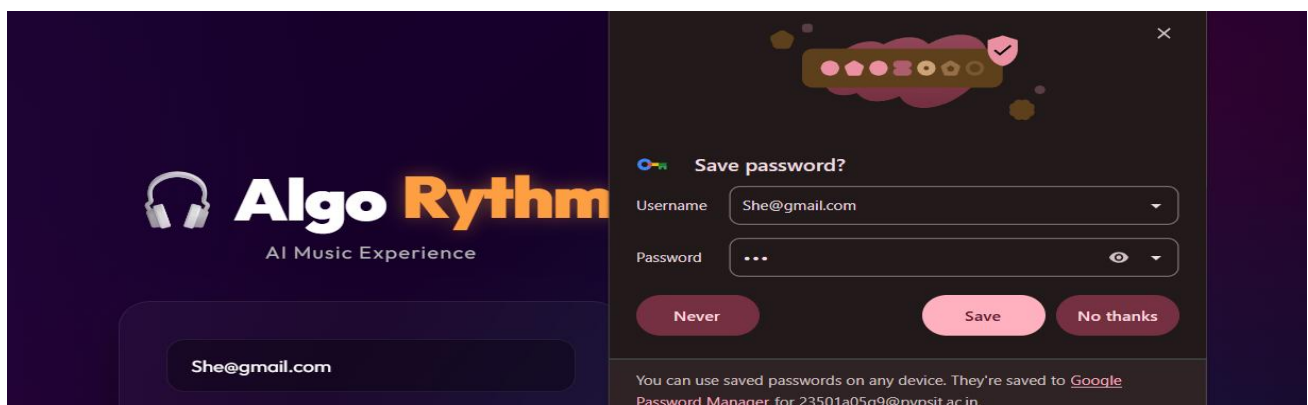
...

Create Account

Already have an account? **Log In**

User registered successfully! You can now login.

Save the Password



Algo Rythm
AI Music Experience

She@gmail.com

Save password?

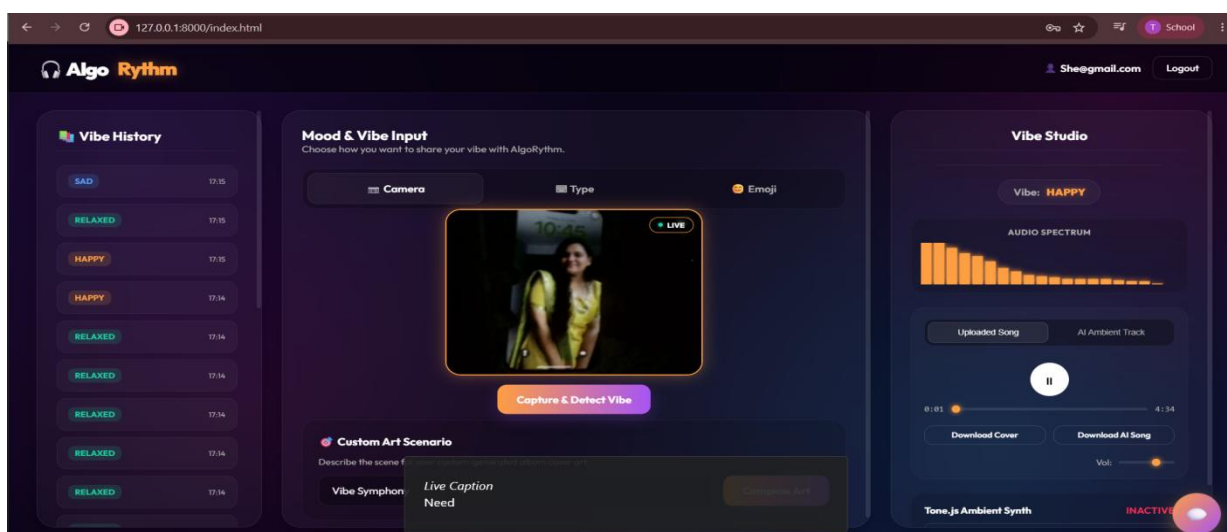
Username: She@gmail.com

Password: ...

Never **Save** **No thanks**

You can use saved passwords on any device. They're saved to [Google Password Manager](#) for 23501a05g9@pvpsit.ac.in.

Webcam Option



Algo Rythm
AI Music Experience

She@gmail.com Logout

Vibe History

- SAD 17:15
- RELAXED 17:15
- HAPPY 17:15
- HAPPY 17:14
- RELAXED 17:14
- RELAXED 17:14
- RELAXED 17:14
- RELAXED 17:14
- RELAXED 17:14

Mood & Vibe Input
Choose how you want to share your vibe with Algorhythm.

Camera Type Emoji

LIVE

Capture & Detect Vibe

Custom Art Scenario
Describe the scene for your custom art.

Vibe Symphony **Live Caption Need** **Generate Art**

Vibe Studio

Viber: **HAPPY**

AUDIO SPECTRUM

Uploaded Song AI Ambient Track

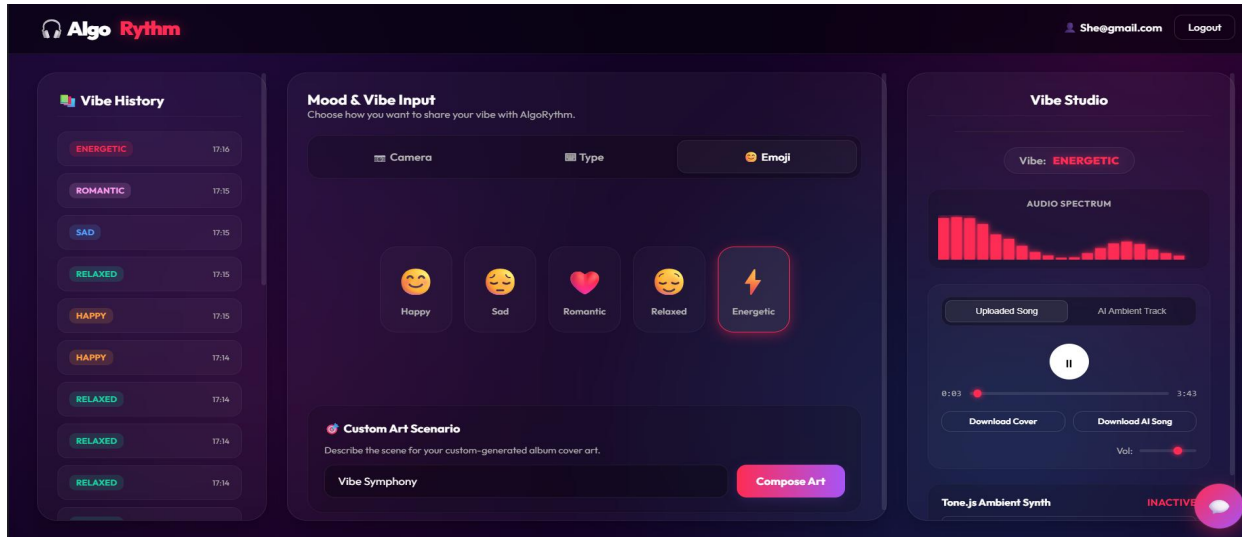
0:01 4:34

Download Cover **Download AI Song**

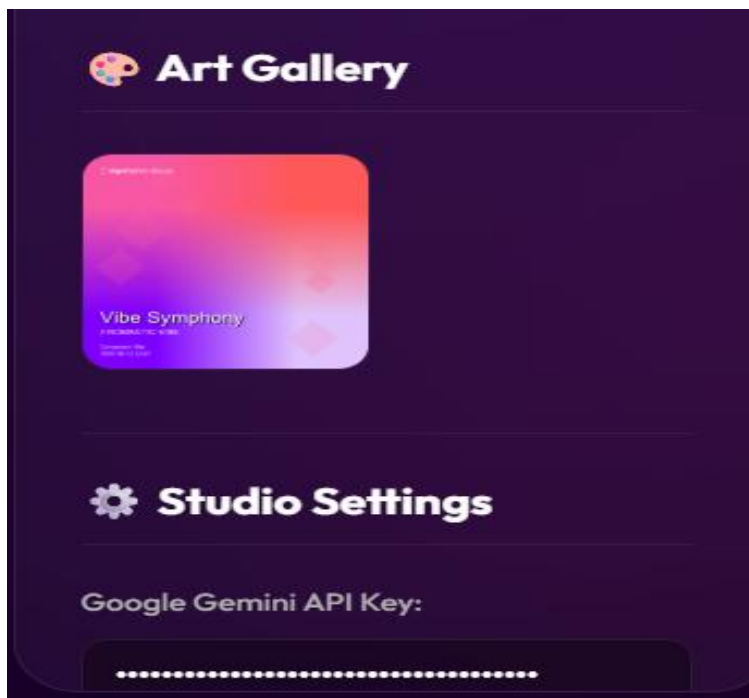
Vol: **INACTIVE**

Tone.js Ambient Synth

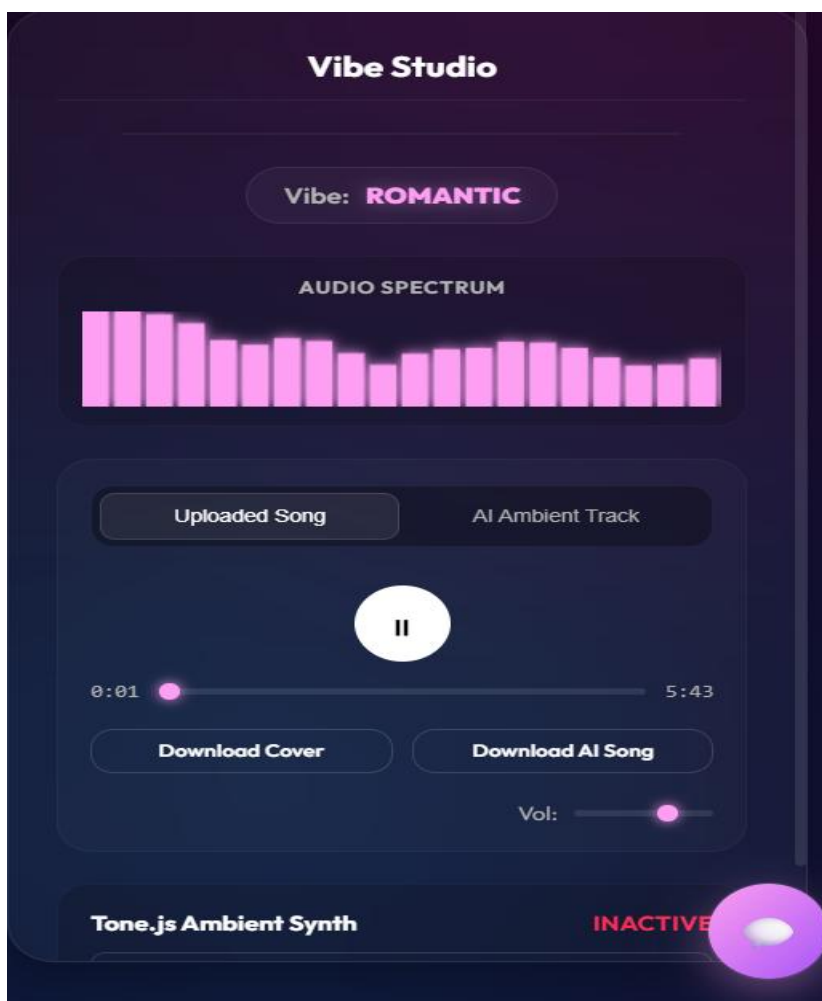
Emoji Option



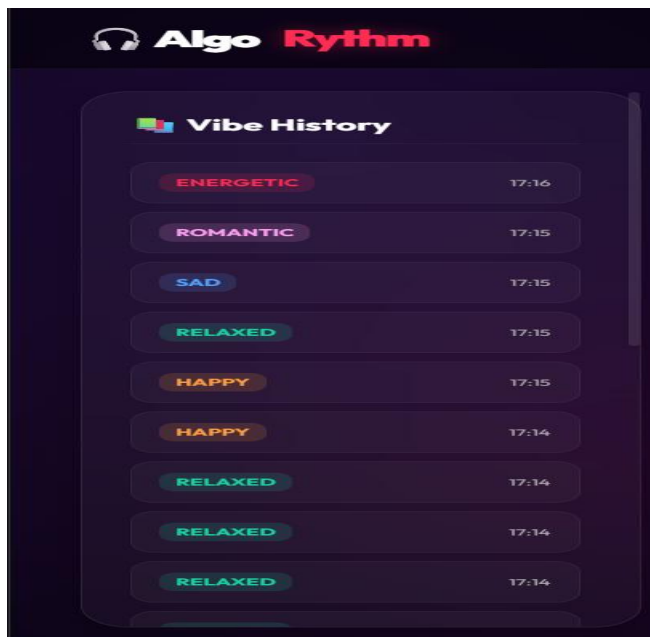
Art Gallery



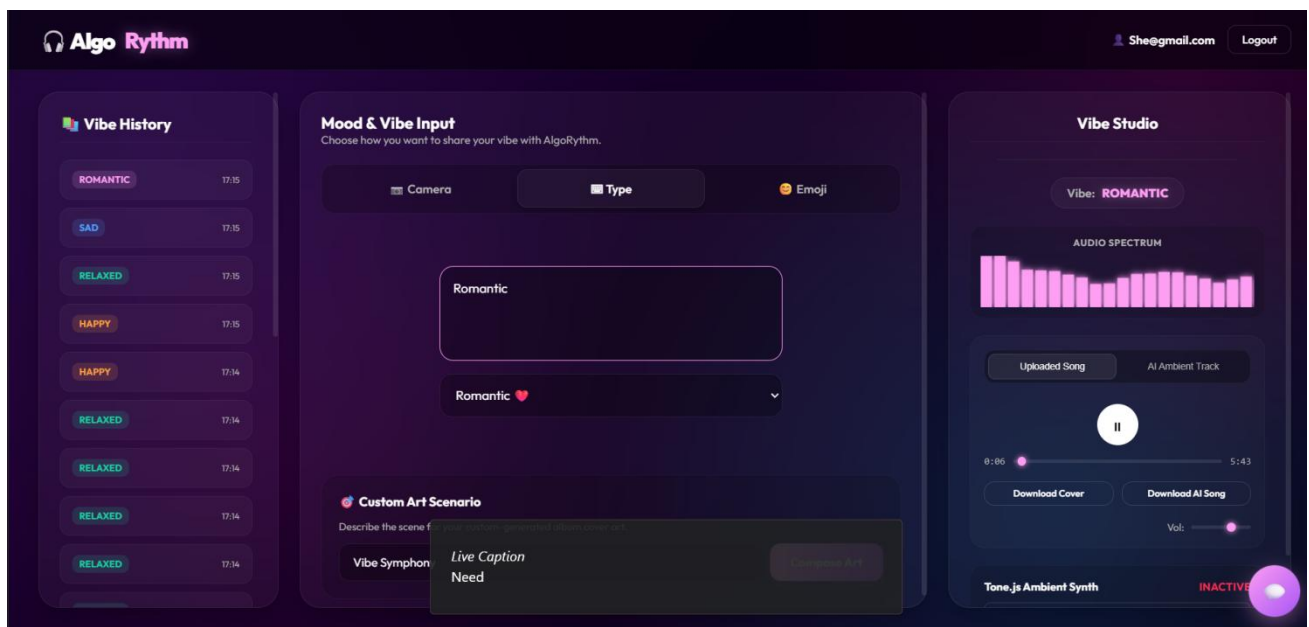
Result



History



Text Option



Future Enhancements

- Add more moods and genres
- Support multiple output formats (MP3, OGG)
- Let users customize instruments
- Save tracks to profiles (authentication support)
- Add rhythm variation and AI-generated lyrics

Conclusion

It blends creativity and code into a truly expressive AI experience. With smooth UX, interactive features, and real-time generation, it demonstrates the potential of generative AI in music. This project serves as a strong foundation for further innovation in music tech.